

Why SSH Keys Win

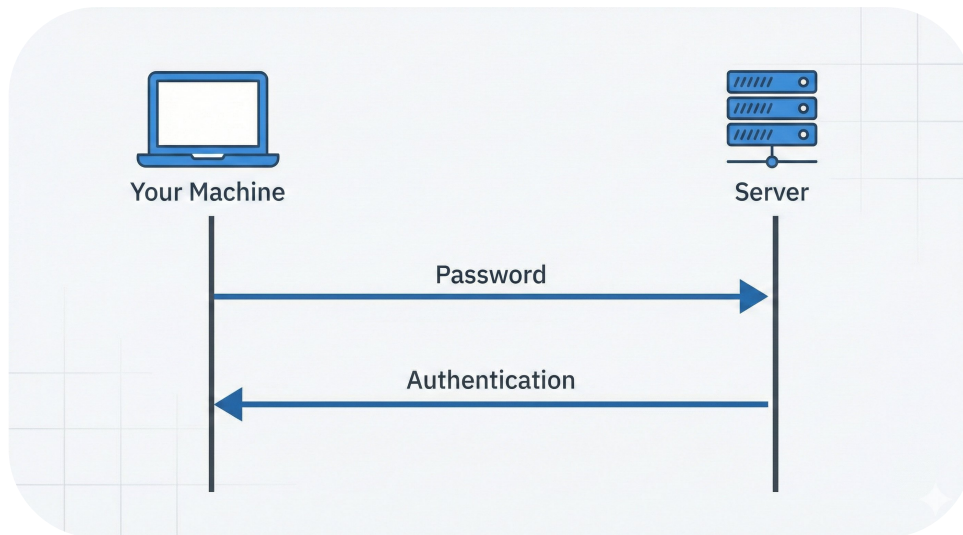
Still using passwords for SSH? I used to. Then I understood why that's like leaving your front door unlocked in a sketchy neighborhood. Here's what changed my mind: **SSH key pairs**. One's public (share freely), one's private (lock it down). No password ever travels over the network—just **cryptography** proving you are who you say you are.

Prepared By: Hosni Zaaraoui

The Problem: Why Passwords Are Broken

➤ Password-Based SSH: The Attack Surface

When you log in with a password:



What can go wrong:

- **Brute force:** Someone runs 1 million password guesses
- **Network sniffing:** Old SSH versions leaked passwords
- **Keyloggers:** Malware on your machine captures your password
- **Credential stuffing:** Your password was in a breach somewhere else
- You use password123 (don't do this)

The math: A 12-character password has about 2^{79} possible combinations. Sounds good, right? But if an attacker can guess a million times per second, they'd crack it in ~8 million seconds (about 3 months).

The Reality

Most servers get attacked by bots trying default credentials: root, admin, ubuntu with passwords like **password**, **123456**, or **nothing**. These bots hammer the login thousands of times. Eventually one sticks.

SSH keys? Immune to this. You can't brute force a 2048-bit RSA key. The math is just too big.

SSH Keys: How They Actually Work (The Simple Version)

SSH keys use **public-key cryptography**, one of the coolest security concepts ever. Here's the intuition:

➤ The Key Pair

When you generate an SSH key, you get two files:

Public Key (**id_rsa.pub**)

- Can be shared freely
- Think of it like a lock you hand out to people
- Anyone can encrypt a message with it
- They cannot decrypt it

Private Key (**id_rsa**)

- Stays on your machine, NEVER shared
- Think of it like your keys to open the lock
- Only you can decrypt messages encrypted with your public key
- Protect this with a passphrase

➤ The Authentication Dance

```
1
2
3   Your machine (has private key)           Server (has public key)
4   |                                         |
5   | 1. "Hey, I'm Ali"                      |
6   +----->|
7   |                                         |
8   | 2. Server: "Prove it. Decrypt          |
9   |    this random challenge I made"       |
10  |<----- (encrypted challenge) -----+
11  |                                         |
12  | 3. Your key decrypts it                 |
13  |    (only your private key can)         |
14  |                                         |
15  +---- (decrypted challenge)----->|
16  |                                         |
17  | 4. Server: "Math checks out.           |
18  |    You must be Ali!"                   |
19  |<----- (connection granted) -----+
```

The beautiful part: Your private key never leaves your machine. The server never sees your password. Just cryptographic proof that you have the right key.

This is bulletproof. An attacker listening on the network sees encrypted challenges and decrypted responses, but can't reverse-engineer your private key from that.

➤ **RSA vs ED25519: Which One?**

You'll see both recommended. Here's the actual difference:

RSA (Older, Still Great)

- Standard since the 1990s
- 2048-bit keys are secure but slower
- 4096-bit keys are more future-proof but overkill for most uses
- Works everywhere (compatibility is excellent)
- Slower key generation

Use RSA if: You need maximum compatibility (old servers, weird systems)

ED25519 (Modern, Preferred)

- Released 2011, adopted widely in the last 5 years
- 256-bit keys, mathematically equivalent to ~3000-bit RSA
- Smaller keys, faster operations
- More resistant to side-channel attacks
- Faster key generation
- Works on basically everything modern

Use ED25519 if: You're not locked into legacy systems (recommended)

Generating Your First Key Pair (5 Minutes)

➤ Step 1: Generate the key

```
ssh-keygen -t ed25519 -C "hosni@gmail.com"
```

What's happening:

- **-t ed25519** — Use ED25519 algorithm (modern, fast)
- **-C "hosni@gmail.com"** — Add a comment (identifies who this key belongs to)

```
File: .id_ed25519
1  -----BEGIN OPENSSH PRIVATE KEY-----
2  b3B1bnNzaC1rZXktdjEAAAAAAAAcmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAAGAAAABBZmygwOa
3  yr7NHKhZThnKtiAAAAEAAAAAAAAAAAAAC3NzaC1lZDI1NTE5AAAAIjIkMwt8SMPo3NbH
4  QswSs0Bgpxrpmc2w59gZcAPJ0U14AAAAo0zI0/98V/vFY3y1+IKT0oDh2qIMcGJwatykd2
5  lWKn/VT+F5Az6ro0yE0z4qkfN3QSnP8ah1G6BJXrR3IGHEow3fis88XFuVs1zC310VjnQY
6  0xsiZ4inSeDF4vQuhUw1xsy59D4n9kW/SFDvuIPM49qJY52IsUxcZNUTkvcSoCeJ5ebwNf
7  LjCVFDBRHKRiLGScsw2aZCwVY1fqAi5+3ULY=
8  -----END OPENSSH PRIVATE KEY-----
```

If you need RSA for compatibility:

```
ssh-keygen -t rsa -b 4096 -C "hosni@gmail.com"
```

```
File: id_rsa
1  -----BEGIN OPENSSH PRIVATE KEY-----
2  b3B1bnNzaC1rZXktdjEAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAACFwAAAAAdzc2gtcn
3  NhAAAAAwEAAQAAAEAgEaj/3BDpW10UPmiIVvgyFwg9PtT4j01cxJ5cvLbUdsRHbn11VAAJ7
4  XFReaFArgdvkNS90SG8z2h8CoPkkK+bpumFFiK0Gb1yTuNeWiLFMS2cMo3rUniPMFvdf3N
5  oo6UN7TJU7bJN+/kZz2SJTklWz68TtG1aJBA3EnFrTfD0rctFdCb91W461y/6pbNb+cywI
6  AvyUpYY/kB/rsJ27e0Z5xB3u0N1SP05hQ5g/r7cJquyRab82DHH+e4sqyYrt5drhD7aatW
7  +hacmMXZPv+C1rVYTsp1TPHBKHQYw3WjGwK5PU6au3ehCD03Py5qf/vxxLVCQ1YgkvEc1r
8  se7M9q9L5V6nj150RnLQ1vCJFSPaXZaomczY5fio5+u0bBSNqstJpUVGaj5WUJqc20VPv4
9  amm6GtY2UXctE15ojpp8nUictGQys50KGuLMnC+jTSWnS9KgjiIavgii5GdVdtn39dVpZob
10 tLoAj8swe70y1wbqUct36evNrMA2fDR1nDLR8UK4GXPoUp/D73t61D7+//TBp6DxVX3CQC
11 nkSZBZYH6LgefP1UU/jZ+tsmPn7Crbsz/LFKqm5avyEhAx9b/Si1rTNhTIAH5Lk8w83+pQ
12 Vm60wF5pAgF9YHEEDXvChxbBM2/R11skS4fGoR5/myknS4DHtcaefrmWpeTKRSL9BYTnGC
13 0AAAdIX9Njc1/TY3IAAAAHc3NoLXJzYQAAAEAgEaj/3BDpW10UPmiIVvgyFwg9PtT4j01cx
14 J5cvLbUdsRHbn11VAAJ7XFReaFArgdvkNS90SG8z2h8CoPkkK+bpumFFiK0Gb1yTuNeWiL
15 FMS2cMo3rUniPMFvdf3Noo6UN7TJU7bJN+/kZz2SJTklWz68TtG1aJBA3EnFrTfD0rctFd
16 Cb91W461y/6pbNb+cywIAvyUpYY/kB/rsJ27e0Z5xB3u0N1SP05hQ5g/r7cJquyRab82DH
17 H+e4sqyYrt5drhD7aatW+hacmMXZPv+C1rVYTsp1TPHBKHQYw3WjGwK5PU6au3ehCD03Py
18 5qf/vxxLVCQ1YgkvEc1rse7M9q9L5V6nj150RnLQ1vCJFSPaXZaomczY5fio5+u0bBSNqs
19 tJpUVGaj5WUJqc20VPv4amm6GtY2UXctE15ojpp8nUictGQys50KGuLMnC+jTSWnS9Kgji
```

➤ Step 2: Choose where to save it

```
hosni PFE 11:00 ssh-keygen -t ed25519 -C "hosni@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/hosni/.ssh/id_ed25519): .id_ed25519
```

Press Enter to use the default location. SSH will create **~/.ssh/** if it doesn't exist.

➤ Step 3: (Optional) Add a passphrase

```
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
```

Should you? Yes, absolutely. A passphrase encrypts your private key file. If your laptop is stolen, the thief can't use your key without the passphrase.

You'll type it once per session (not per login, thanks to ssh-agent—more on that later).

Done!

Two files appear:

- **~/.ssh/id_ed25519** — Your private key (chmod 600, never shared)
- **~/.ssh/id_ed25519.pub** — Your public key (safe to share)

Adding Your Key to a Server (The Right Way)

You have your key pair. Now you need to tell the server about your public key.

➤ Method 1: Using `ssh-copy-id` (Easiest)

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server.com
```

This:

1. Reads your public key
2. Connects to the server (prompts for password, just this once)
3. Appends it to `~/.ssh/authorized_keys` on the server
4. Sets the correct permissions automatically

Done. Now you can log in without a password:

➤ Method 2: Manual (If `ssh-copy-id` doesn't work)

1. On your machine, copy your public key

```
cat ~/.ssh/id_ed25519.pub
```

2. SSH to the server (password login, **last time**)

```
ssh user@server.com
```

3. On the server, add it to `authorized_keys`

```
mkdir -p ~/.ssh
```

```
echo "YOUR_PUBLIC_KEY_HERE" >> ~/.ssh/authorized_keys
```

```
chmod 600 ~/.ssh/authorized_keys
```

```
chmod 700 ~/.ssh
```

Critical: The permissions matter. If `authorized_keys` is world-readable, SSH ignores it for security. Must be 600 (owner only).

Why This Stops 99% of Brute-Force Attacks

➤ The Bot's Perspective

Every day, automated scripts probe millions of servers trying to log in as **root** with **password**:

```
ssh root@target.com -p 22 # With password guessing
```

With password auth: After 1000 attempts, maybe one succeeds.

With SSH keys only: The command fails immediately. There's no password to guess. The key either works or it doesn't. No middle ground.

From the attacker's perspective, key-based-only servers are invisible to brute-force tools. They move on.

The Numbers

According to honeypot data, the vast majority of SSH attacks are password brute-force:

- 99%+ use password guessing
- Less than 1% involve actual key theft or sophisticated attacks
- Disabling passwords stops almost everything

This is why security best practice is: **disable password authentication entirely.**

➤ On Your Server: Disable Passwords (Recommended)

```
sudo nano /etc/ssh/sshd_config
```

Find and change:

```
PasswordAuthentication yes
```

To:

```
PasswordAuthentication no
```

```
PubkeyAuthentication yes
```

Save, restart SSH:

```
sudo systemctl restart ssh
```

Now only key-based auth works. Brute-force attacks become pointless.

ssh-agent: So You're Not Typing Passphrases Constantly

You added a passphrase to your key (good!). But typing it every time you SSH is annoying.

Enter **ssh-agent**: A program that holds your decrypted key in memory for your current session.

➤ How it works

Start ssh-agent (usually automatic on login)

```
eval "$(ssh-agent -s)"
```

Add your key to the agent

```
ssh-add ~/.ssh/id_ed25519
```

Prompts for passphrase, caches it

Now SSH uses the cached key

```
ssh user@server.com # No passphrase prompt!
```

```
ssh another@server.com # Still no prompt!
```

The agent holds the key until you log out or explicitly remove it:

```
ssh-add -d ~/.ssh/id_ed25519 # Remove from agent
```

```
ssh-add -D # Remove all keys from agent
```

Real-World SSH Key Workflow

➤ Day 1: Setting up on a new machine

Generate key

```
ssh-keygen -t ed25519 -C "you@gmail.com"
```

Add to agent

```
ssh-add ~/.ssh/id_ed25519
```

Copy public key to GitHub, servers, etc.

```
cat ~/.ssh/id_ed25519.pub
```

➤ Every day: Just use SSH

```
ssh user@server.com
```

```
git clone git@github.com:yourname/repo.git
```

```
scp file.txt user@server.com:/home/user/
```

The agent handles everything. You never type a passphrase because it's cached.

➤ **When you get a new machine or lose your key**

Generate a new key on the new machine

```
ssh-keygen -t ed25519 -C "you@gmail.com"
```

Add public key to all servers

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server.com
```

Add to GitHub/GitLab settings

```
# (Paste from: cat ~/.ssh/id_ed25519.pub)
```

Done.

Common SSH Key Mistakes And How to Avoid Them

➤ Sharing your private key

Never, ever email or upload your private key. If someone has it, they are you (until you revoke it).

Do: Only share the public key (.pub file)

➤ Bad permissions on private key

```
ls -l ~/.ssh/id_ed25519
-rw-rw-r-- 1 you you 1679 Nov 24 id_ed25519
```

World-readable private key = useless. SSH will refuse to use it.

Fix:

```
chmod 600 ~/.ssh/id_ed25519
```

➤ No passphrase on your key

```
ssh-keygen -t ed25519 -C "you@gmail.com"
```

Hit Enter when asked for passphrase

Now your key is unencrypted. If your laptop is stolen, the thief has everything.

Do: **Always add a passphrase.** ssh-agent caches it, so you're not typing constantly.

➤ Multiple keys without naming them

You have `id_ed25519`, `id_rsa`, `id_special...`

SSH gets confused which one to use

Instead, name them clearly:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_github -C "github"
```

```
ssh-keygen -t ed25519 -f ~/.ssh/id_server -C "production  
server"
```

```
ssh-keygen -t ed25519 -f ~/.ssh/id_aws -C "aws instances"
```

Then tell SSH which key to use (via `~/.ssh/config`—covered in the next post).

➤ Using the same key everywhere

One key for GitHub, AWS, production, your homelab? If one place gets breached, they all compromise.

Do: **Use different keys for different places.** Not hard, and it's defense in depth.

SSH Key Security Checklist

- Generated ED25519 or 4096-bit RSA keys
- Added a passphrase to your private key
- Set chmod 600 on your private key
- Never shared your private key
- Added public key to servers with correct `authorized_keys` permissions
- Using `ssh-agent` so you're not retyping passphrases
- Named multiple keys clearly (`id_github`, `id_server`, etc.)
- Considered disabling password authentication on your servers

Why This Matters

Keys aren't just more secure. They're:

- **Bulletproof against brute force** — An attacker can't guess a 256-bit key
- **Convenient** — ssh-agent caches passphrases, no constant typing
- **Scriptable** — You can automate deployments, backups, etc. with key auth
- **Auditable** — You can see which machines have your public key
- **Revocable** — Remove your public key from `authorized_keys` and you're locked out instantly (if the key is compromised)

Once you go keys-only, passwords feel primitive.

Quick Troubleshooting

Check what keys the agent has

```
ssh-add -l
```

Verbose SSH (shows which key was used)

```
ssh -v user@server.com
```

Test your public key on a server

```
ssh -i ~/.ssh/id_ed25519 user@server.com
```

See what keys SSH will try

```
ssh -G user@server.com | grep -i identity
```